

Réponses aux questions du TD

Question 2. À quoi servent respectivement JUnit, Hamcrest, Mockito et JaCoCo ? Lequel n'est pas une bibliothèque de test à proprement parler ?

Réponse :

- **JUnit** : il fournit les annotations (`@Test`, `@BeforeEach`, `@ParameterizedTest`...), le moteur d'exécution (JUnit Platform) et les assertions de base (`assertEquals`, `assertThrows`...).
- **Hamcrest** : C'est une bibliothèque de *matchers* elle sert à écrire des assertions plus expressives et composables (`assertThat(x, is(...))`) que celles fournies nativement par JUnit.
- **Mockito** : une bibliothèque de doublures de test, elle permet de créer des stubs/mocks/spies automatiquement (`mock(...)`, `when(...).thenReturn(...)`, `verify(...)`) au lieu de les écrire à la main.
- **JaCoCo** : ce n'est *pas* une bibliothèque de test à proprement parler, mais un plugin Gradle de mesure de couverture de code : il instrumente le bytecode pour savoir quelles lignes/branches ont été exercées par les tests, sans participer à l'écriture ou à l'exécution des tests eux-mêmes.

Question 2 : Comparer le test et le code : quelle information apparaît dans les deux ? Pourquoi est-ce le signe qu'il reste du travail ?

Réponse :

La chaîne `"org.acme"` apparaît à la fois dans le test (valeur attendue) et dans le code (valeur codée en dur dans `Gav.parse`). Le paramètre `coordinate` n'est jamais utilisé : la méthode ne parse rien, elle renvoie toujours la même constante. Le test passe par coïncidence, pas parce que le comportement est correct — un seul cas ne permet pas de distinguer un vrai parsing d'une valeur en dur. Il faut trianguler avec un second cas (Q6) pour forcer une vraie implémentation.